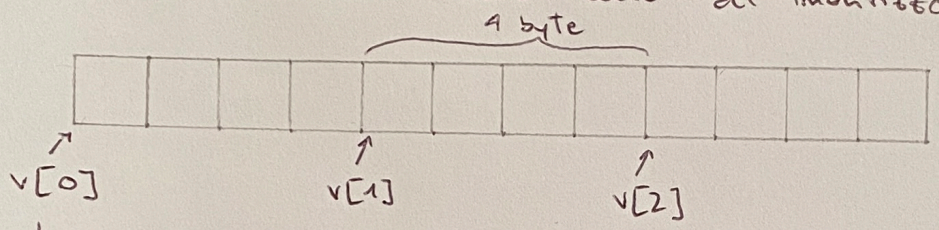
Gli array e le variabili vengono dichiarati in una cella all'interno della memoria. ~~del~~ Per rintracciare la cella viene fornito un indirizzo che caratterizza ogni cella di memoria, in base al tipo di dato verranno occupate più o meno celle.

x esempio:

int v[7] = { .... } size of int = 4 byte

⇒ v[0] = base
v[1] = base + 1 · size of int = base + 4
v[2] = base + 2 · 4 = base + 8

⇒ tutti i 7 numeri interi che compongono l'array v saranno disposti l'uno a fianco all'altro a partire dal primo che sarà alla cella all'indirizzo "a", il secondo che sarà alla cella all'indirizzo "a+4" ecc...



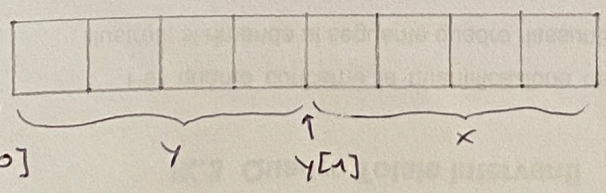
viene dichiarato il primo int del nostro array

Se il tipo è double ⇒ lo spazio necessario sarà di 8 byte

Nel caso del codice della figura 1 cosa succede?
Vengono (riga 9-10) inizializzate 2 nuove variabili

int x = 1; size of int = 4 byte
float y = 1.1; size of float = 4 byte

Guardando gli indirizzi notiamo che y viene dichiarato 4 byte "prima" di x → compilazione dello stack avviene dall'alto verso il basso



Eseguendo il rigo 12 (&y)[1] = 0; sto dicendo che il valore di y all'indice 1 è = 0 ma l'indirizzo in cui viene ~~salvato~~ dichiarato ciò è lo stesso di x ⇒ y[1] = 0 andrà a sovrascrivere x = 1